

Ingegneria del Software

Ingegneria del software e Processi di sviluppo

Prof. Paolo Giorgini

A.A. 2026/2027

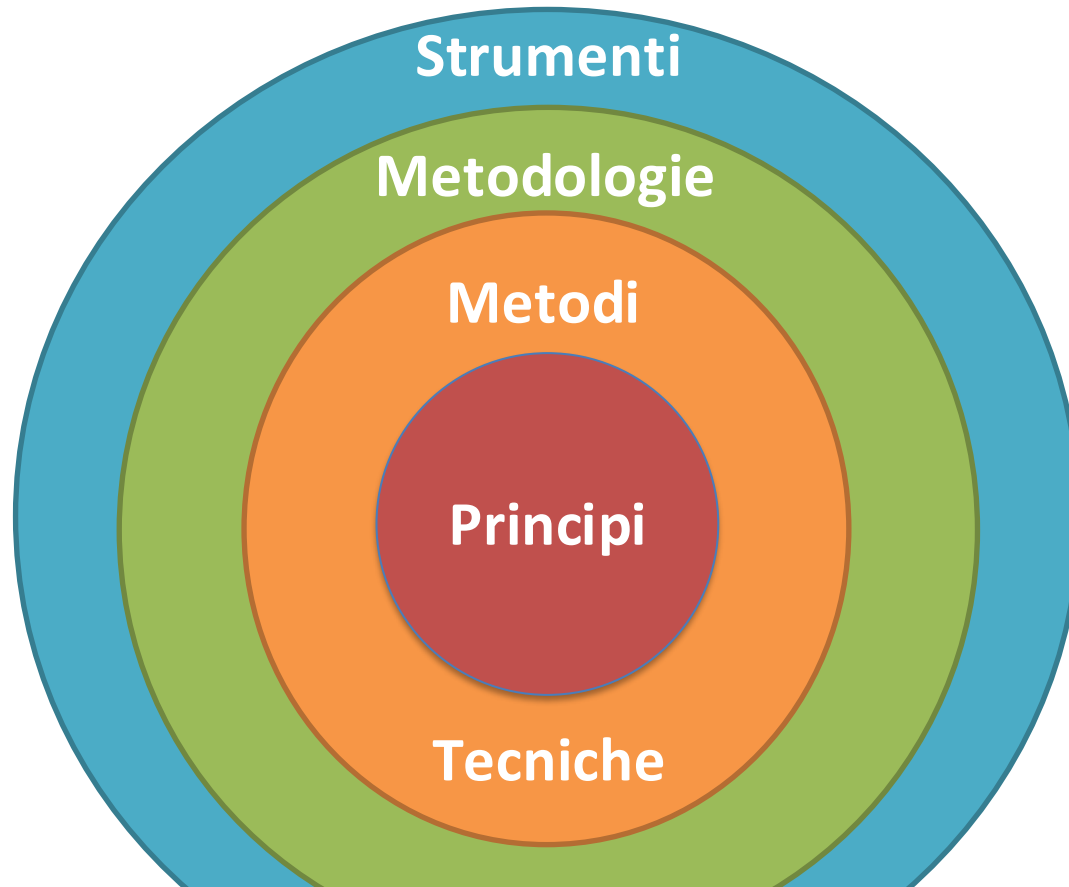
Alcune definizioni di IS

- Fritz Bauer [1969]: un punto di partenza
 - L'ingegneria del software è l'istituzione e l'**impegno di principi ingegneristici ben fondati**, allo scopo di ottenere in **modo economico software affidabile ed efficiente**
- Scott Whitmire
 - Più che una disciplina o insieme di conoscenze, è un **modo di affrontare un problema**
- IEEE [1993]
 - (1) applicazione di una strategia sistematica, disciplinata e misurabile allo sviluppo, esercizio e manutenzione del software; (2) studio di strategie di cui al punto (1).

Idea chiave: l'Ingegneria del Software non riguarda solo la programmazione. Riguarda il modo sistematico con cui comprendiamo, specifichiamo, progettiamo, realizziamo, verifichiamo ed evolviamo sistemi software.

Questo principio resta valido anche quando parte del codice viene prodotto da AI Coding Agents

IS come tecnologia stratificata



**Gli AI Coding Agents sono nuovi strumenti.
Non sostituiscono principi, metodi e processi.**

Principi

- Rigorosità e formalità
- Separazione dei problemi
- Modularità
- Astrazione
- Previsione dei cambiamenti
- Generalità
- Incrementalità

AI-assisted development

Principio classico	Nell'AI-assisted development
Rigorosità	Specification, acceptance criteria, quality gates
Separazione	Feature decomposition, task decomposition
Modularità	Component boundaries, task scope
Astrazione	Requirements, models, specifications
Cambiamenti	Traceability, versioning, living specifications
Incrementalità	Iterative implementation and verification

L'AI cambia gli strumenti e parte del processo, non elimina i principi dell'Ingegneria del Software.

Rigorosità e Formalità

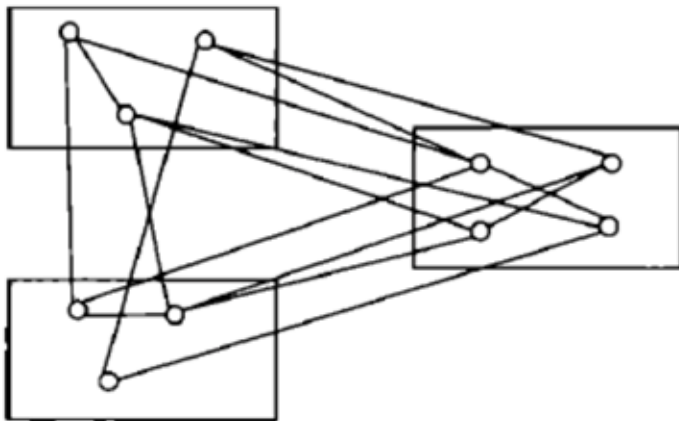
- Ingegneria del software
 - Creativa ma ... richiede sistematicità
- Rigorosità e formalità
 - Necessario complemento a creatività per aumentare il livello di confidenza
- Esempi
 - Prova formale di correttezza
 - Documentazione precisa

Separazione dei problemi

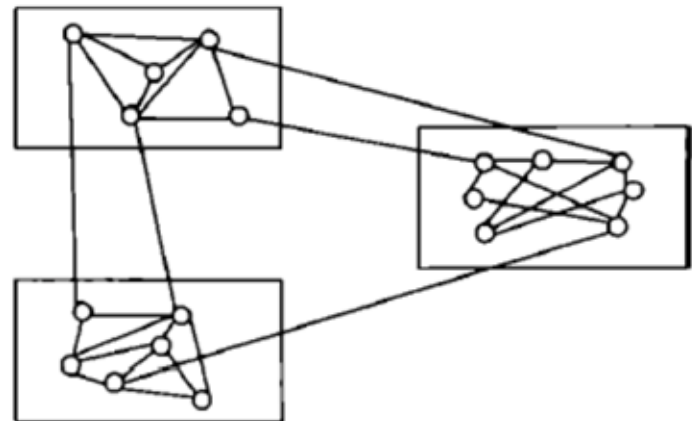
- Separare i problemi per gestire la complessità
 - Dividi et impara
- Esempi
 - Diverse fasi del processo
 - Front-end / back-end di un web app
 - Diverse tecniche di test per diversi problemi

Modularità

- Dividere un sistema in pezzi semplici: moduli
 - Alta coesione
 - Basso accoppiamento



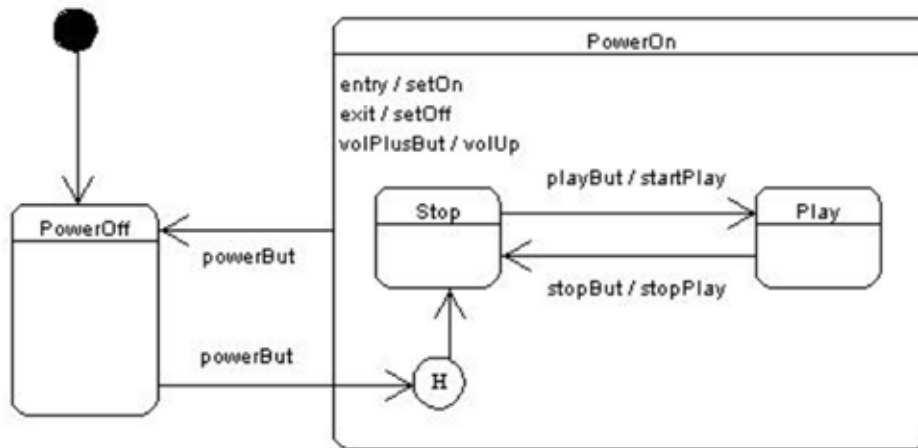
(a)



(b)

Astrazione

- Ignorare i dettagli
 - Premessa per i modelli



```

class CPlayer {
    CPlayerState state; // state object
    AbsOnState onHistory; // for History state
    // References for all the state objects
    PowerOff offState; PowerOn onState;
    Stop stopState; Play playState;
    CPlayer() { // constructor
        // create state objects only once
        offState = new PowerOff(this); onState = new PowerOn(this);
        stopState = new Stop(this, onState);
        playState = new Play(this, onState);
        state = offState; // sets the default state
        onHistory = stopState; // setting the history for the first time }

    void setState(CPlayerState st) { // setting new state
        state = st;
        //sets the most recent active substate of PowerOn state
        if(state.equals(onState)) { onState.substate = onHistory;
        state.entry(); // executes the entry action }
    // Delegates incoming Events to concrete state class
    void powerBut() { state.powerBut(); }
    void volPlusBut() { state.volPlusBut(); }
    void playBut() { state.playBut(); }
    void stopBut() { state.stopBut(); }
    // Actions becomes methods in Super Context class
    void setOn() { ..... }
    void setOff() { ..... }
    void volUp() { ..... }
    void startPlay() { ..... }
    void stopPlay() { ..... }
}

class CPlayerState { // Abstract Class
    CPlayer cp; // reference to the context object
    ..... // Declaring abstract methods
}

class PowerOff extends CPlayerState {
    void powerBut() {
        exit(); // executes the exit action of current state
        cp.setState(cp.onState); // change to PowerOn state }
}

class PowerOn extends CPlayerState {
    AbsOnState substate; // reference for nested statechart
    void entry() {
        substate.entry(); // executes entry action of active substate
        cp.setOn(); // executes entry action }
    void exit() {
        cp.setoff(); // executes exit action
        cp.onHistory = substate; // adjusting the history node }
    void volPlusBut() { // Internal Transition
    
```

Previsione dei cambiamenti

- Capacità di affrontare l'evoluzione
 - Gestire quello che potrebbe cambiare in futuro
 - Prima e dopo la realizzazione
- Esempio
 - Gestione delle configurazioni
 - Mantiene traccia delle relazione fra i vari moduli e le evoluzioni delle configurazioni dei vari moduli
 - La versione 3.0 e' stata derivata dalla 2.95 in funzione di certi tipi di cambiamenti, la 2.95 e' compatibile con la 2.7 di un altro modulo

Generalità

- Risolvere il problema generale, non solo l'istanza
 - Riusabilità: possiamo riusare la soluzione generale in istanze e situazioni diverse
 - Attenzione ai costi
 - Generalizzare ha un costo e chiaramente non vogliamo avere la soluzione per tutti i problemi

Incrementalità

- Procedere a passi successivi
- Esempi
 - Sviluppo per prototipi incrementali
 - Iterazioni successive con il cliente che vede crescere il sistema
 - Aumento dei costi / tempi

Processo di sviluppo

- Stabilisce quando e come qualcuno fa cosa (artefatti), per raggiungere un determinato obiettivo



Attività → Ruoli → Artefatti → Controlli → Output

Con l'AI cambia anche il “chi”: alcune attività possono essere delegate a coding agents, ma il processo deve continuare a definire responsabilità e criteri di controllo.

Scelta e adattamento

- Legato al problema, team, cultura aziendale
- Differenze tra modelli
 - Flusso delle attività, dei task e loro interdipendenza
 - Attività di controllo del progetto
 - Dettaglio e rigore del processo
 - Coinvolgimento degli stakeholders
 - Autonomia del team
 - Prescrizione dei ruoli e loro organizzazione
 - grado di automazione/delega;
 - ruolo di AI tools e coding agents nel processo
- Modelli agili
 - Alta adattabilità

Non esiste un processo “migliore” in assoluto: deve essere adattato al contesto, al rischio e al grado di autonomia che concediamo agli strumenti

Valutazione del processo

Valutare il processo per determinati livelli di qualità

- ripetibile
- controllabile
- misurabile
- verificabile
- migliorabile

Esempi

- SCAMP: Standard CMMI Assessment Method for Process Improvement
- CBA IPI: CMM-based Appraisal for Internal Process Improvement
- SPICE: ISO/IEC15504
- ISO 9001:2000 for software

Cosa descrive un modello di processo?

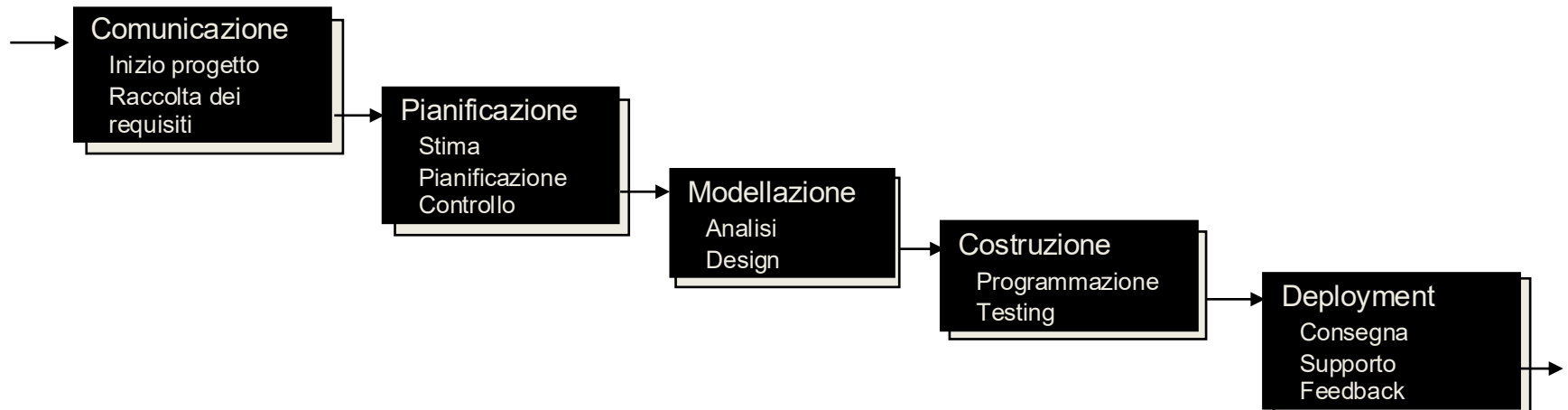
Un modello di processo definisce:

- ordine delle attività
- feedback e iterazioni
- artefatti prodotti
- responsabilità
- criteri per passare da una fase alla successiva

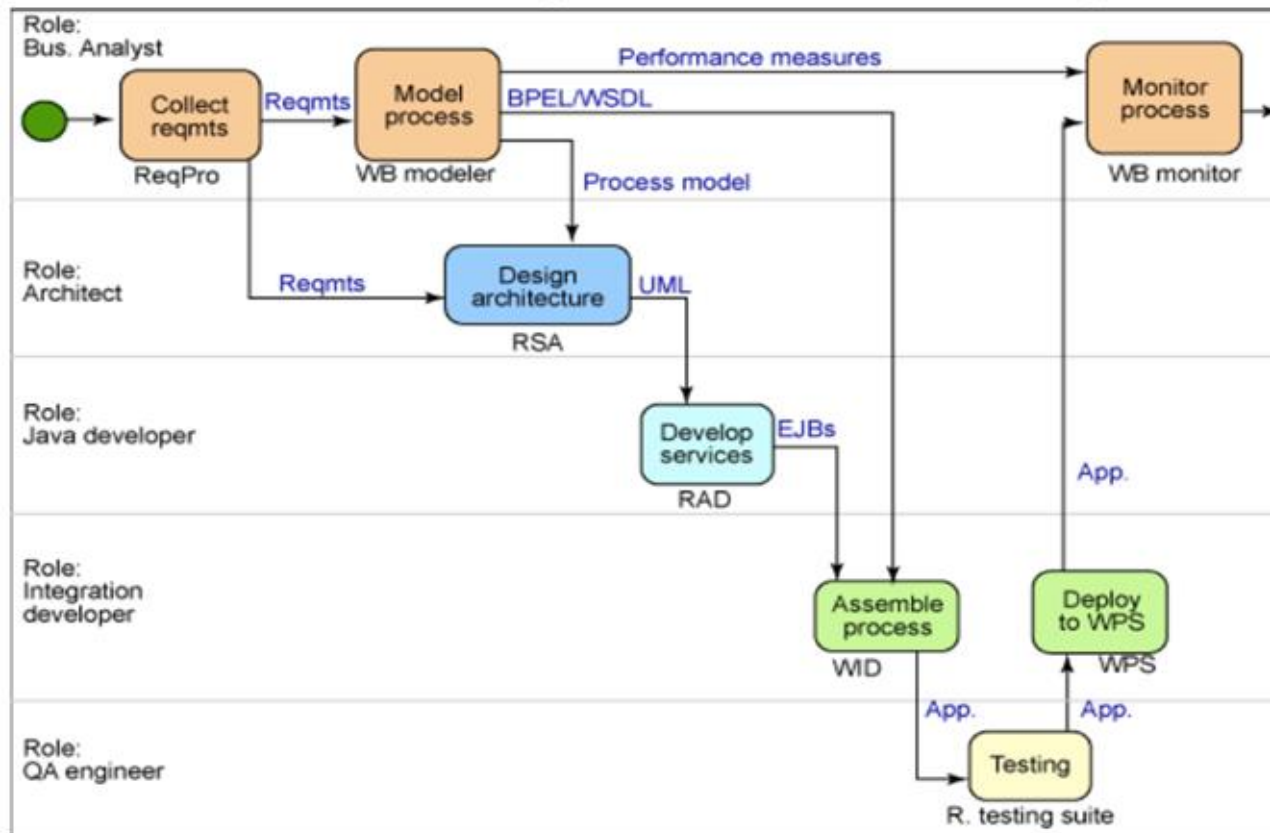
La differenza tra Waterfall, Incremental, Agile e approcci più recenti non è solo “l'ordine delle fasi”, ma **come gestiscono feedback, cambiamento e controllo**

Modello di Processo a cascata

- Requisiti noti e il lavoro può procedere in maniera lineare
 - adattamenti o estensioni ben definite ad un sistema esistente
- Modello Waterfall (classico)



Esempio processo a cascata (IBM WebSphere)

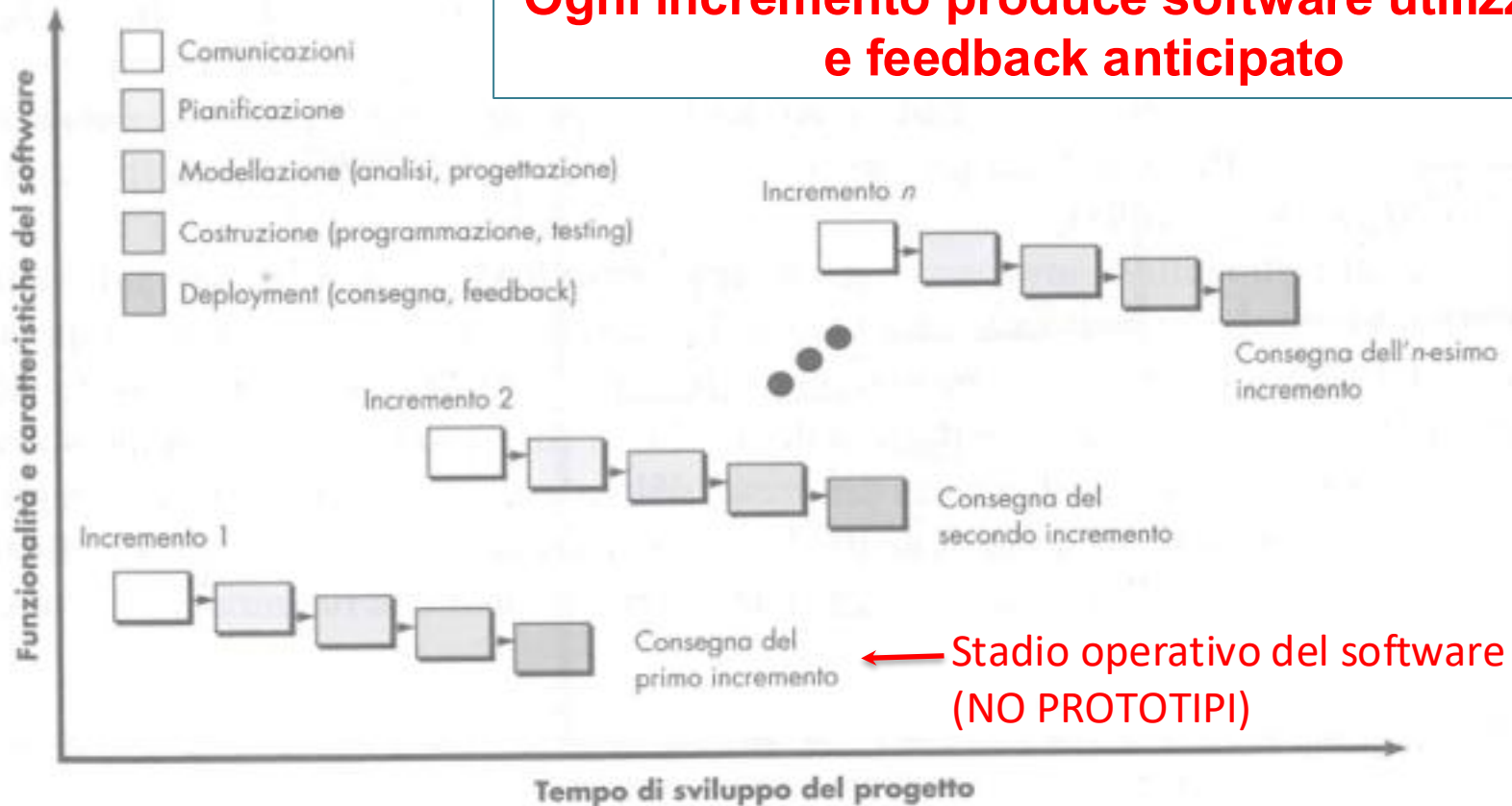


Punto di forza: controllo e chiarezza delle fasi.

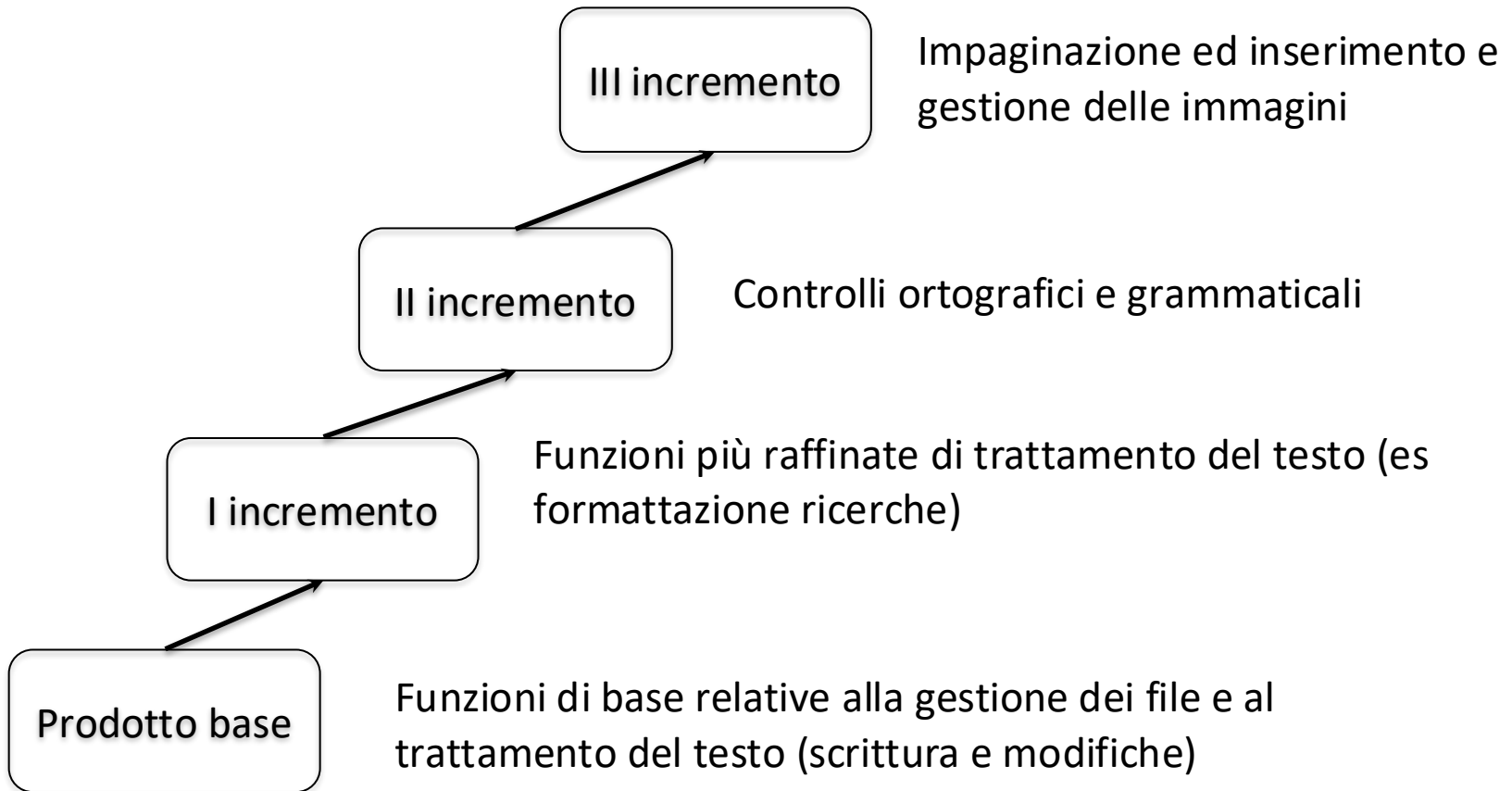
Limite: i problemi scoperti tardi costano di più.

Modello di processo incrementale

Ogni incremento produce software utilizzabile e feedback anticipato

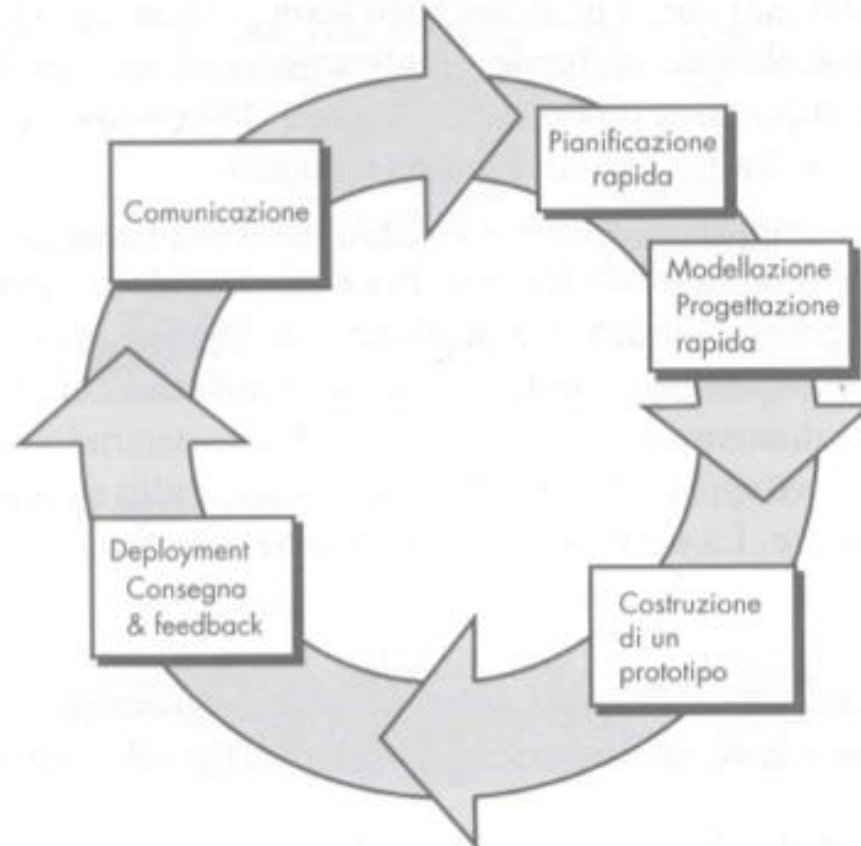


Es programma di elaborazione di testi



Modello a prototipi

Quando il cliente ha un bisogno legittimo ma non riesce a fornire indicazioni precise sui dettagli, il primo passo consiste nello sviluppo di un prototipo



Prototipi (def)

- Def ISO 13407
*una rappresentazione di un prodotto o di un sistema, o di una sua parte, che, anche se in qualche modo limitata, può essere utilizzata a scopo di **valutazione***
- un prototipo non deve necessariamente essere un sistema funzionante, spesso può essere utile anche un semplice modello “finto” (mock-up)

Jeff Hawkins, l'inventore del Palm Pilot, il primo PDA di successo, inizialmente tenne con sé un modellino in legno dello strumento, ovviamente non funzionante, fingendo di tanto in tanto di inserirvi o di leggervi delle informazioni



Che fine fa il prototipo?

- Il prototipo è un sistema preliminare
 - Spesso non utilizzabile, lento, troppo esteso, difficile da utilizzare
- Meglio ricominciare da capo
- Rischi:
 - Il cliente ha per le mani una versione del software che sembra essere quella finale
 - Gli sviluppatori scendono a compromessi per sviluppare rapidamente il prototipo
 - Qualità scadente

Wireframe vs Mock-up

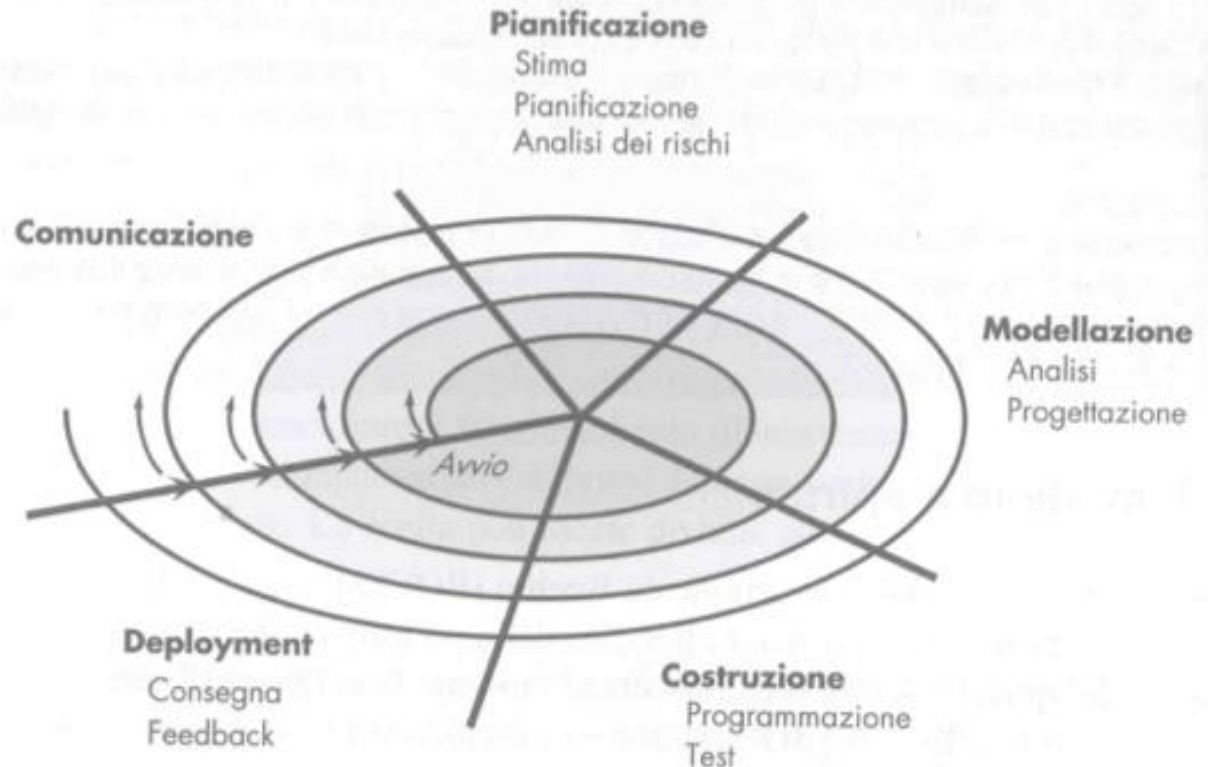


- Wireframe: il focus è nella **user experience design** (trovare la migliore user experience).
 - Crea una bozza grafica fatta su un foglio di carta fatto a matita oppure utilizzando software specifici (es. Balsamiq)
 - Rappresenta la base dell'idea (spesso in bianco e nero)
- Mock-up: rappresenta la fase successiva in cui gli elementi del design entrano in gioco (colore, caratteri, immagini, etc) – si hanno più dettagli per dimostrare il **look and feel** (quello che sarà il prodotto).

Modello a spirale

Abbina la natura iterativa delle prototipizzazione e gli aspetti controllati e sistematici del modello a cascata

L'iterazione non serve solo a sviluppare più velocemente: serve a ridurre progressivamente l'incertezza



Perché i processi diventano iterativi?

Perché all'inizio non conosciamo tutto:

- i requisiti possono essere incompleti
- alcune decisioni tecniche sono incerte
- il cliente chiarisce meglio i bisogni vedendo risultati intermedi
- test e feedback fanno emergere nuovi problemi
- Il processo diventa un ciclo di apprendimento, non solo una sequenza di attività

Dal feedback umano al feedback strutturato

Tradizionalmente

- prototipo → cliente → feedback → revisione

Con AI-assisted development

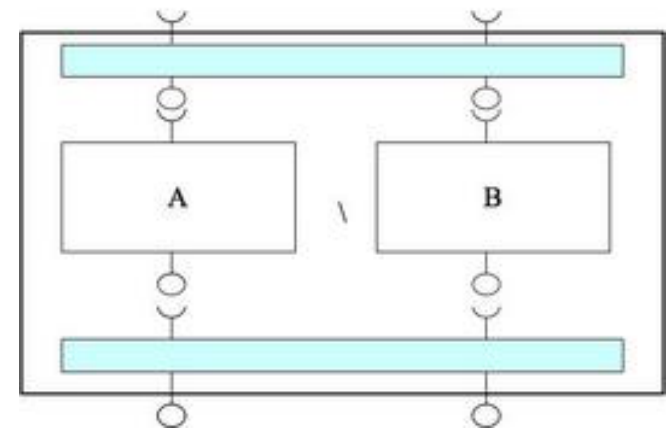
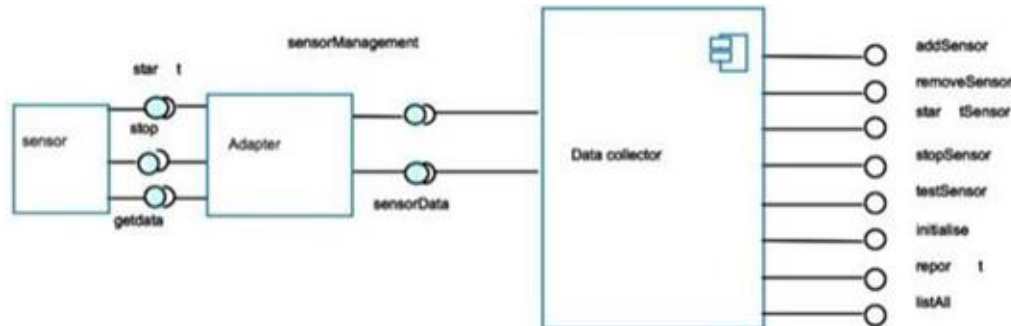
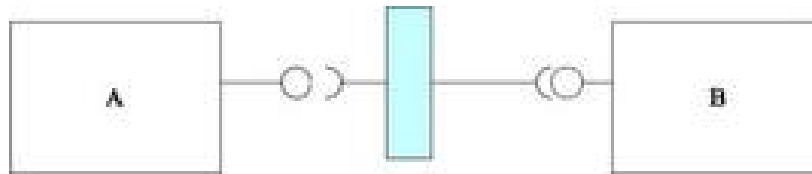
- specification → implementation → test/eval → verification → revisione
- Il feedback resta centrale, ma deve diventare più esplicito e verificabile quando parte dell'implementazione è delegata a un agente

Model-Driven Development

- Modello, rappresentazione del sistema
 - Una forma particolare di prototipo del sistema
- Creazione di un modello di alto livello che evolve con la progettazione e lo sviluppo
 - Modello come guida per lo sviluppo
 - Strumenti per la generazione automatica del codice e dei test case
- Model-Driven Architecture (OMG)
 - Uso della notazione UML
- L'artefatto centrale non è il codice, ma un **modello che guida la costruzione del sistema**

Sviluppo a componenti

- Uso di componenti software con funzionalità mirate con interfacce ben definite



responsabilità ben definite
interfacce esplicite
basso accoppiamento
possibilità di riuso

Composizione dei componenti

- I componenti più semplici risultano essere i più riusabili ma non riescono a risolvere problemi complessi
- Una composizione di componenti può raggiungere requisiti più specifici e complessi
 - è un processo che consente di ottenere un sistema attraverso l'assemblaggio di componenti
 - Spesso è necessario scrivere del codice integratore (glue code) per:
 - Rendere compatibili i formati degli input e degli output dei vari componenti
 - Coordinare i vari componenti
- Nuovi problemi: compatibilità, coordinamento e correttezza delle interazioni
- Non basta che i singoli componenti siano corretti, deve essere corretta anche la loro composizione

Valutazione dei componenti (esempio)

- Nel 1996, il razzo Ariane 5 (ESA) si distrusse durante il primo test di volo poichè il sistema ne perse il controllo dopo 37 secondi
 - un componente riusato dal software di Navigazione Inerziale sviluppato per Ariane 4
 - una eccezione di overflow non gestita, la quale però non poteva verificarsi nell'Ariane 4, a causa della ridotta potenza rispetto al 5



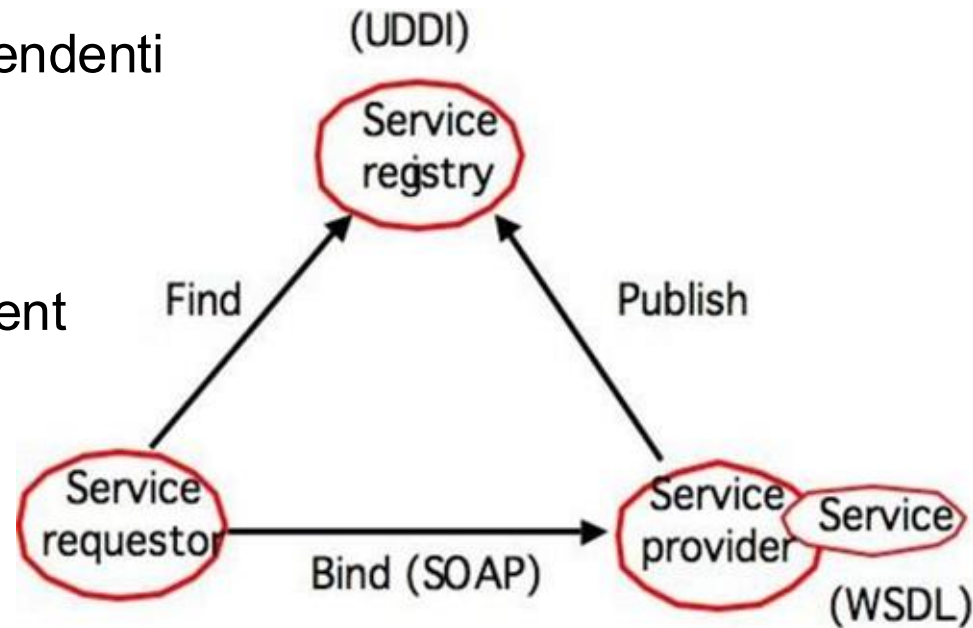
Un componente corretto in un sistema può non esserlo in un altro

Architetture Orientate ai Servizi (SOA)

Sistema composto da servizi indipendenti
Interfacce/contratti espliciti
Comunicazione tramite rete

Vantaggio: evoluzione e deployment indipendenti

Costo: maggiore complessità di integrazione e coordinamento



Più il sistema è distribuito, più diventano importanti contratti, osservabilità e verifica delle interazioni

Un pattern ricorrente in IS

- Quando cresce la complessità, introduciamo artefatti e confini più espliciti
 - **Models → Components → Services → Specifications**
- Il principio non cambia: rendere esplicite le decisioni che altrimenti resterebbero implicite
- Con i Coding Agents, rendere espliciti contesto, vincoli e criteri di verifica diventa ancora più importante
 - Specification → Plan → Tasks → Agent → Verification

Metodologie Agile

- Per **metodologia agile** (o leggera) o **metodo agile** si intende un particolare metodo per lo sviluppo del software che coinvolge quanto più possibile il committente, ottenendo in tal modo un'elevata reattività alle sue richieste
- Esistono un certo numero metodologie Agile
 - Agile alliance, organizzazione no-profit creata allo scopo di diffonderle
 - <http://www.agilealliance.org>

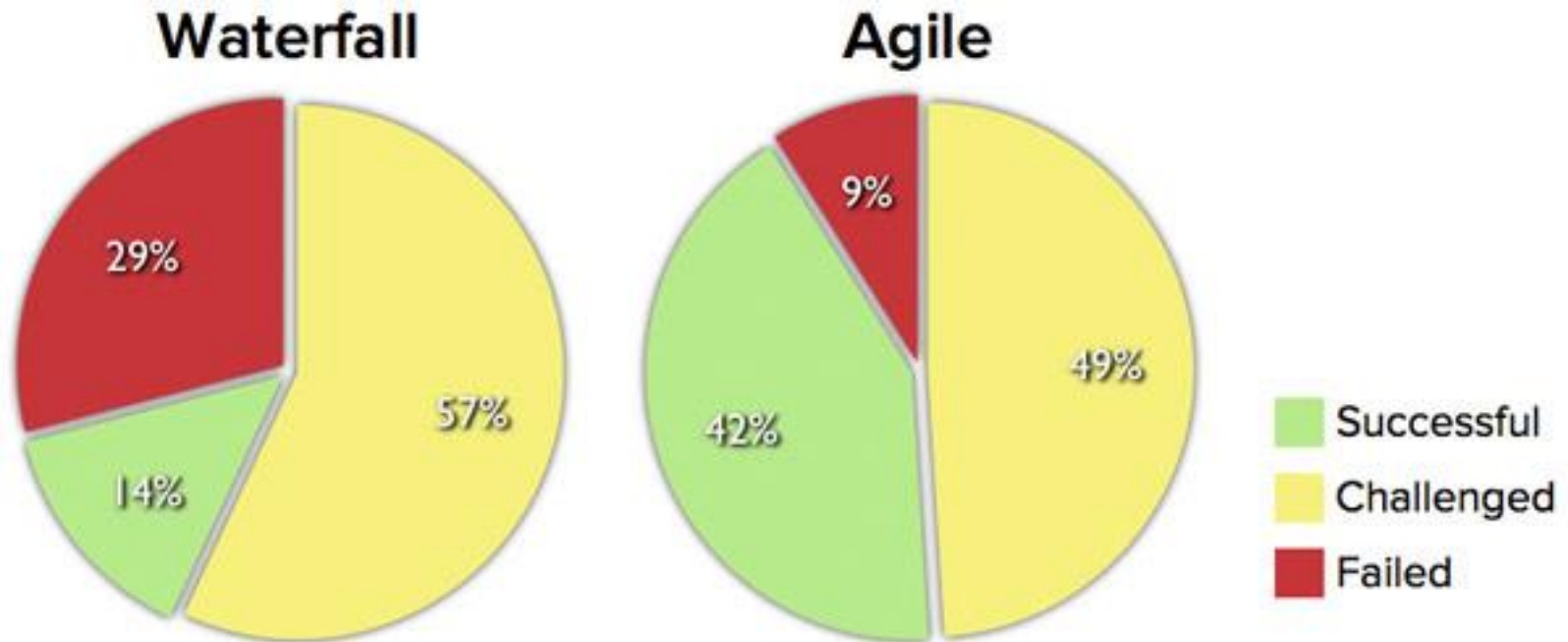


L'approccio Agile

- **Agile** indica una famiglia di approcci allo sviluppo software basati su:
 - iterazioni brevi;
 - feedback frequente;
 - collaborazione con gli stakeholder;
 - adattamento al cambiamento;
 - rilascio incrementale di valore.
- Ridurre il tempo tra una decisione e il feedback che ci dice se quella decisione era corretta



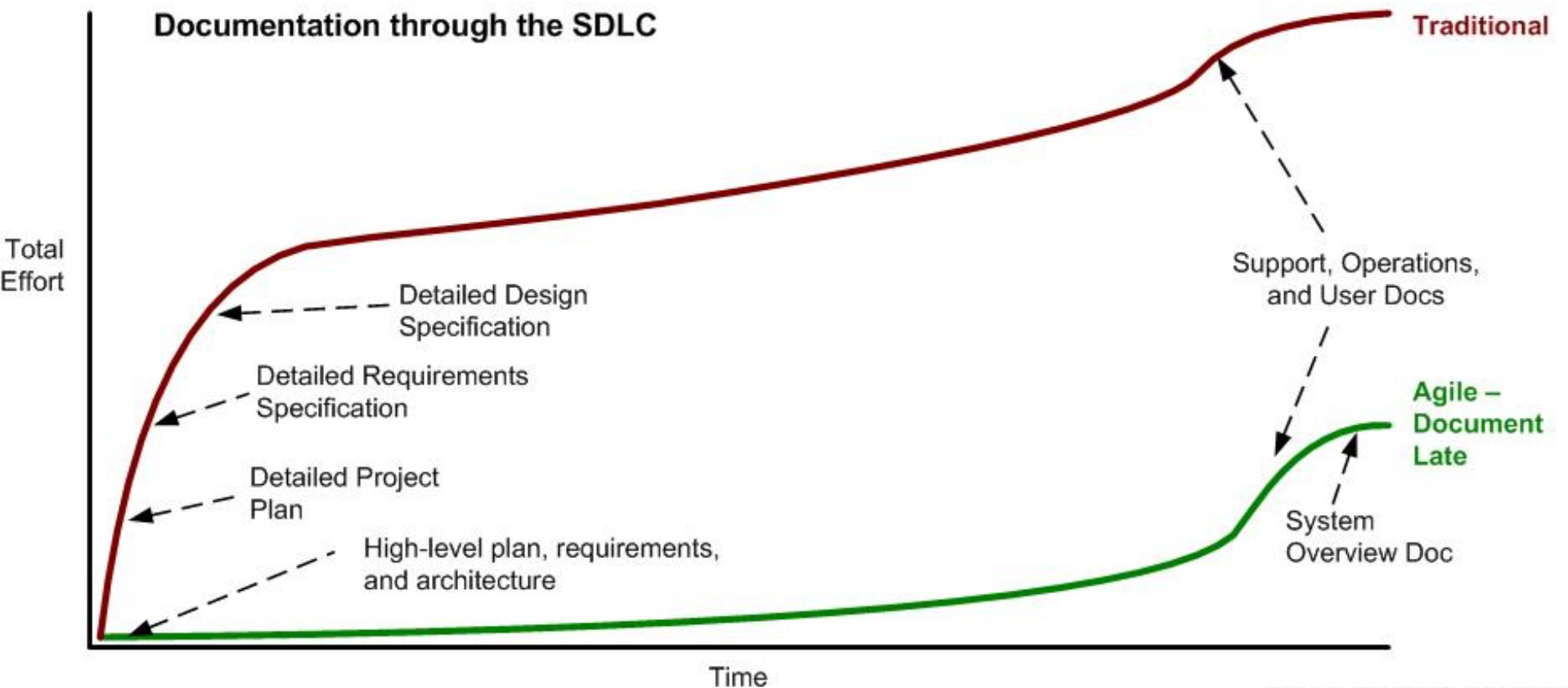
Agile vs Waterfall



Source: The CHAOS Manifesto, The Standish Group, 2012.

Waterfall cerca di ridurre l'incertezza prima dello sviluppo.
Agile la gestisce attraverso feedback e iterazioni durante lo sviluppo.

Documentazione



Copyright 2006-2014 Scott W. Ambler

Agile significa meno documentazione?

- La domanda non è “documentare o non documentare”, ma:
 - Quale informazione deve essere esplicita?
 - Per chi?
 - Per quanto tempo?
 - Per quale decisione o verifica?

Documentazione a basso valore

- prodotta per obbligo
- Duplicata
- rapidamente obsoleta
- non usata dal team

Documentazione ad alto valore

- guida decisioni
- mantiene contesto
- supporta testing e verifica
- evolve con il software

Con gli AI Coding Agents, parte della documentazione diventa anche input operativo per l'implementazione

Processi Agili

- Guidati dalla descrizione del cliente di che cosa gli serve: scenario
 - una descrizione di possibili interazione tra utenti e sistema (descrizione del funzionamento)
- Sviluppano software in maniera iterativa con forte enfasi sulle attività di costruzione
 - molteplici *incrementi software*
 - Adattamento ai cambiamenti
 - Ogni iterazione è un ciclo:
 - Intent → Implementation → Feedback → Adaptation

Agile vs IS

Non si deve pensare erroneamente che l'agilità dia la licenza di improvvisare le soluzioni. E' necessario un processo ed è fondamentale la disciplina

Non si tratta di scegliere fra agilità e ingegneria del software (classica) ma di definire un approccio agile all'ingegneria del software

Molti concetti della metodologia agile non sono altro che adattamenti dei migliori concetti di ingegneria del software

Agile riduce la burocrazia, non la disciplina

Agile + AI: cosa cambia?

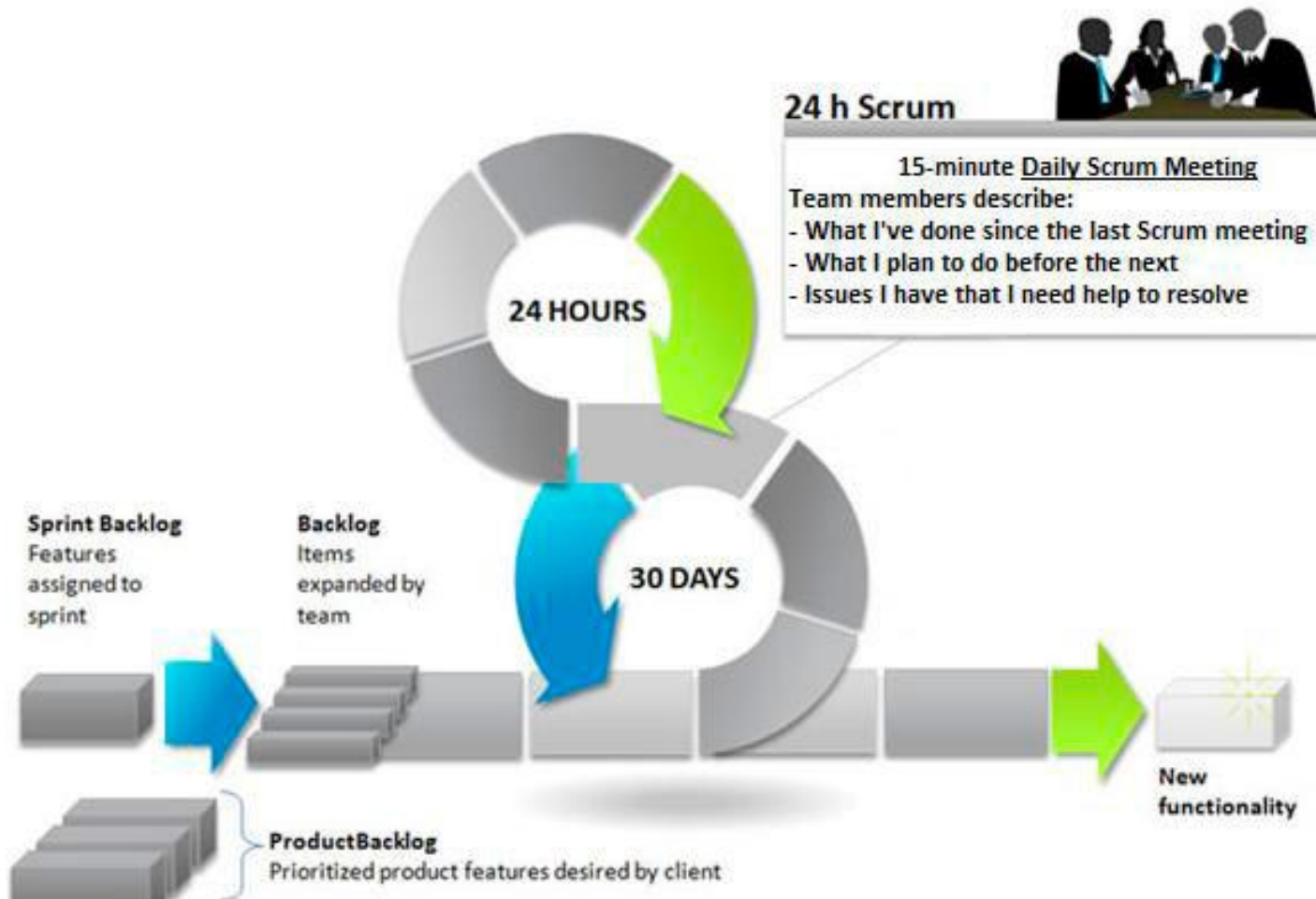
- Agile assume che possiamo correggere frequentemente il software grazie al feedback
- Con gli AI Coding Agents aumenta drasticamente la velocità con cui possiamo produrre modifiche
- Ma se possiamo produrre codice più velocemente, possiamo anche produrre errori più velocemente
- Diventano ancora più importanti:
 - intent esplicito
 - task piccoli
 - acceptance criteria
 - test
 - human review
 - traceability

Il modello Scrum (mischia)

- Insieme di **pratiche e regole** che permettono di ridurre l'**overhead** organizzativo
 - Piccoli team di lavoro organizzati
 - Produzione di frequenti incrementi software
 - Costante attività di testing e documentazione
 - Divide il progetto in blocchi rapidi di lavoro (**sprint**) alla fine dei quali viene consegnata una versione del sistema al cliente, indica come definire i dettagli del lavoro da fare nell'immediato futuro (**backlog**) per averne una definizione estesa, organizza riunioni giornaliere del team di sviluppo (**daily scrum**) per verificare cosa si è fatto e cosa si farà

Scrum

- Attività strutturali
 - Requisiti, analisi, design, evoluzione e consegna



Seminario 19 ottobre

Kanban Simulation

Dr. York Rössler



Extreme Programming (XP)

- Kent Beck, Ward Cunningham e Ron Jeffries (1996)
- Approccio Object-Oriented per lo sviluppo
- **4 valori e 12 pratiche**
 - I 4 valori su cui si fonda questa metodologia sono: **semplicità, comunicazione, testing e coraggio**
 - Le 12 "prassi consolidate" possono essere raggruppate in quattro aree fondamentali: **feedback a scala fine, processo continuo, comprensione condivisa, benessere dei programmatori**

Extreme Programming (XP)

- **Continuous integration**
 - L'integrazione delle modifiche nel sistema viene **fatta frequentemente**, in modo da limitare i conflitti che il nuovo codice (e il codice di test) dovesse dare
 - Procedura strettamente sequenziale: solo uno sviluppatore alla volta integra le sue modifiche con il sistema, testa e rilascia il codice
- **Refactoring o design improvement**
 - refactoring del codice cambiando l'architettura, in modo da renderlo più semplice e generico
- **Small releases**
 - consegna del software avviene tramite frequenti rilasci di funzionalità che creano del valore concreto

XP: Planning (1)

- Creazione di un insieme di **user story**
 - Descrivono le caratteristiche e le funzionalità del software (descritta dal cliente – scheda indice)
 - Il cliente assegna un valore (priorità) a ciascuna user story
 - I membri del team XP valutano ciascun user story e le assegnano un costo (settimane di sviluppo)
 - Se richiede più di tre settimane viene chiesto al cliente di dividerla in casi più semplici
 - Nuove user story possono essere scritte in qualsiasi momento
 - I clienti ed il team XP collaborano per decidere il modo in cui raggruppare le user story per la realease successiva
 - Accordo di massima (user story, data di consegna etc)
 - Il team XP decide se: (1) implementare immediatamente tutte le user story; (2) implementare prima quelle a valore più alto; (3) implementare prima le user story più rischiose

User Story Examples

- A user wants access to the system, so they find a system administrator, who enters in the user's First Name, Last Name, Middle Initial, E-Mail Address, Username (unique), and Phone Number.
- **Risk: Low Cost: 2 points**

Front of Card

173

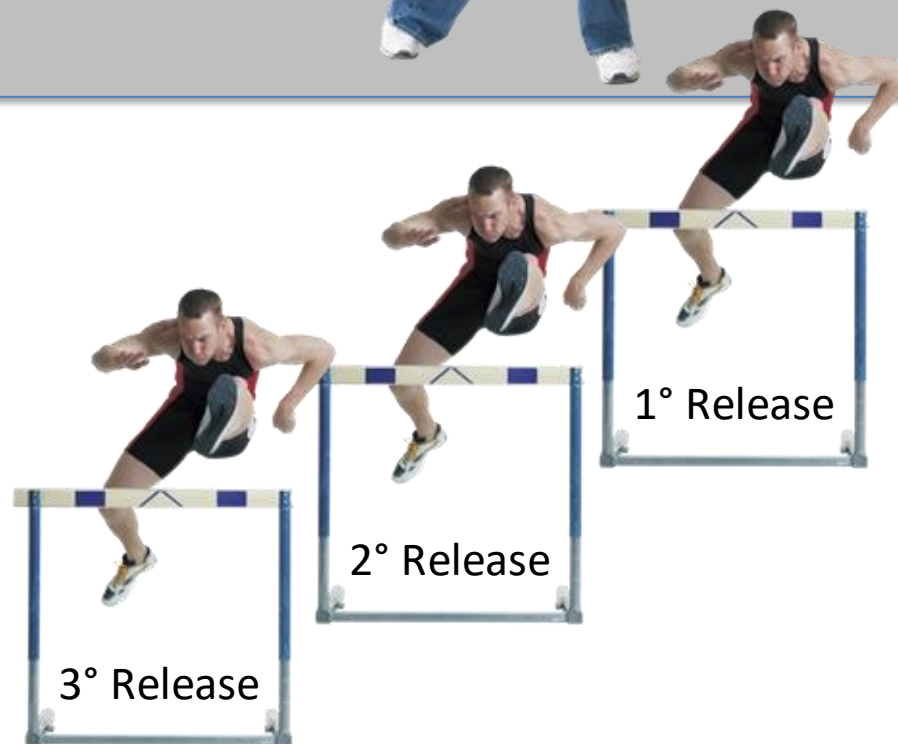
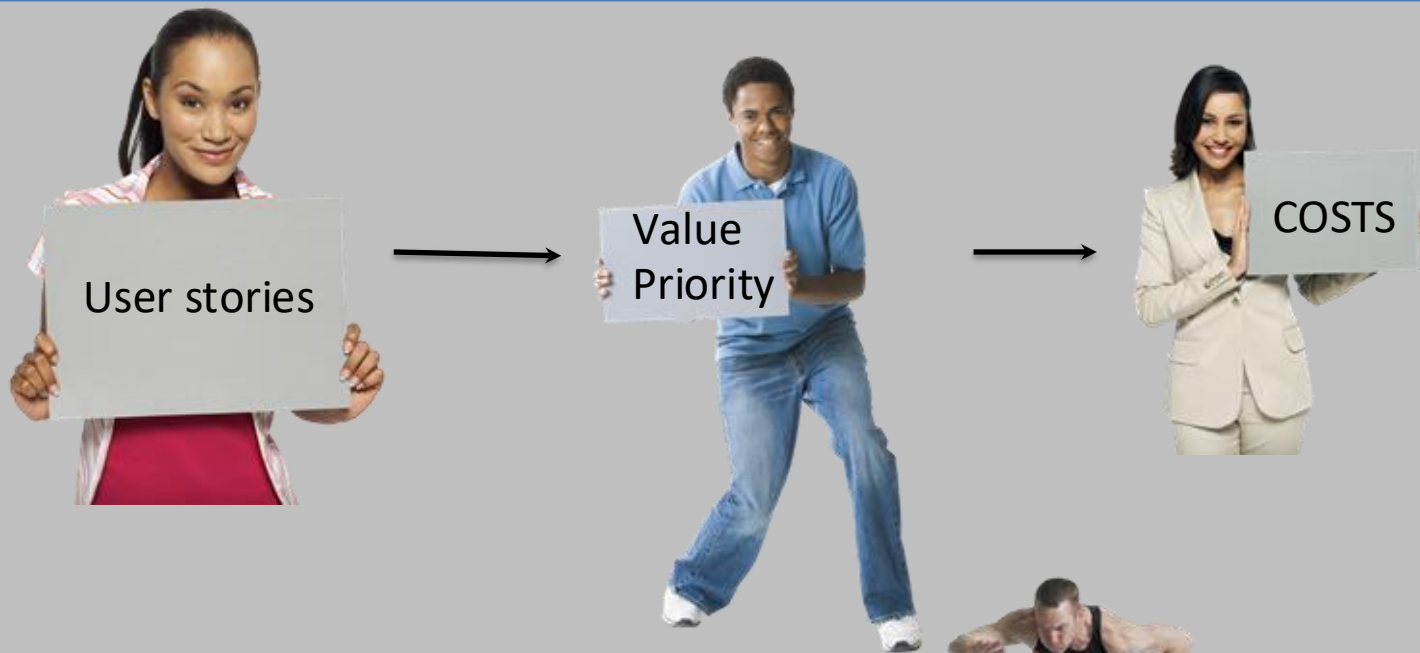
As a student I want to purchase a parking pass so that I can drive to school

Priority: ~~High~~ Should
Estimate: 4

Back of Card

Confirmations:

~~The student must pay the correct amount~~
One pass for one month is issued at a time
The student will not receive a pass if the payment isn't sufficient
The person buying the pass must be a currently enrolled student.
The student may only buy one pass per month.



XP: Programmazione

- Sviluppo di Unit Test
 - Metteranno alla prova ognuna delle user story
 - Il programmatore si potrà quindi concentrare su ciò che deve essere implementato per passare tali test – non viene aggiunto nulla di superfluo
 - Pair programming (programmazione a coppie)
 - Due persone (con ruoli distinti) che collaborano alla stessa workstation per sviluppare il codice
 - Garantisce qualità e soluzione rapida a problemi
 - Il loro lavoro è poi integrato con quello degli altri (team di integrazione) – integrazione continua
 - Evita problemi di compatibilità e di interfacciamento

XP: Testing

- Utilizzo di strumenti di supporto per l'automazione degli unit test
 - Integration e validation test eseguiti quotidianamente (testing suite universale)
 - Indicazione continua dei progressi e individuazione più efficace delle problematiche
 - Acceptance test (o customer test) sono semplificati e si concentrano sulle funzioni e le caratteristiche globali del sistema

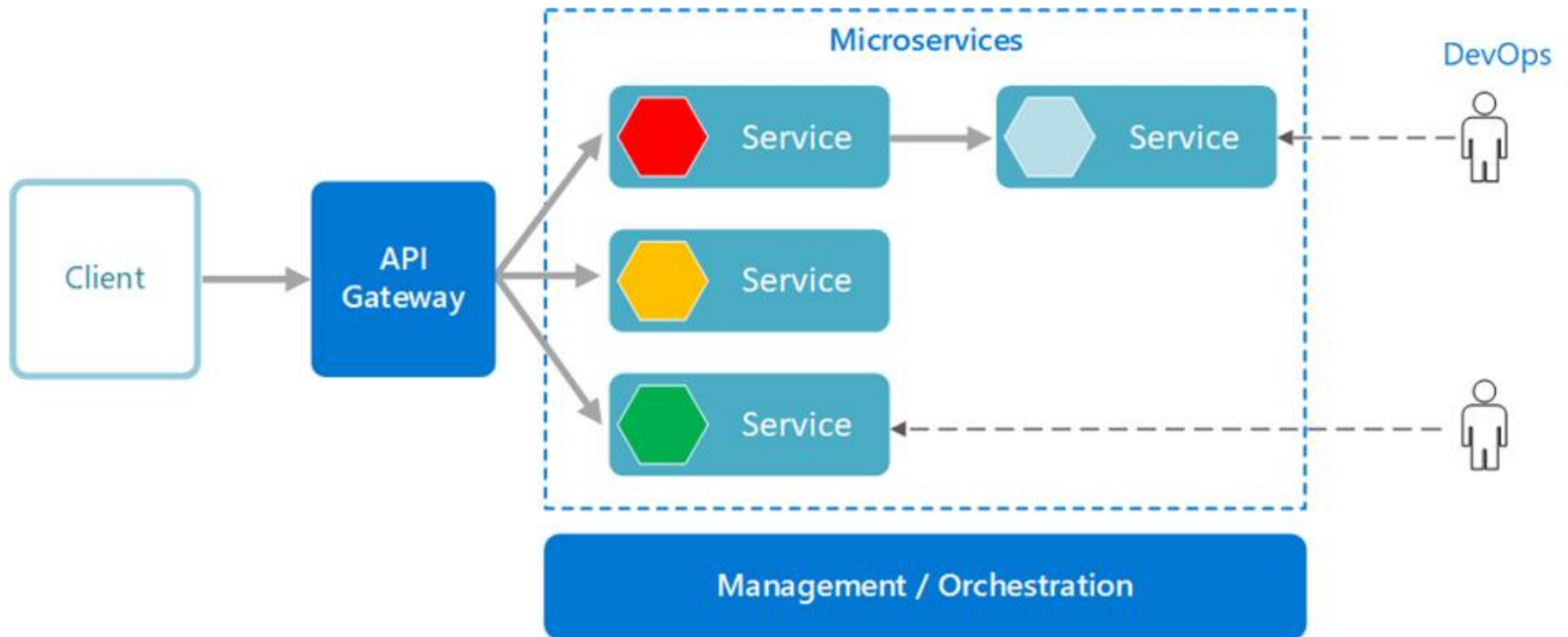
DevOps

- In realtà strutturate i componenti software vengono installati in infrastrutture grandi e complesse
- Bisogna coordinare il lavoro di chi scrive il software (**Development**) con quello di chi gestisce l'infrastruttura (**Operations**).
- Il termine **DevOps** (development+operations) indica una metodologia che consente di gestire in modo rapido ed efficiente questa interazione.
- Si basa su:
 - Virtualizzazione dei datacenter (o cloud)
 - Strumenti di automazione (IaC – infrastructure as code)
 - Metodologie di lavoro uniformi e condivise fra sviluppatori e sistemisti (di tipo agile).

Modularizzazione e microservizi

- La modularizzazione è essenziale nello sviluppo software
- L'architettura che si sta più diffondendo è quella a microservizi (**Microservice Architecture** o **MA**)
 - E' adatta per lo sviluppo di **grandi applicazioni** che hanno la necessità di scalare ed evolversi velocemente sfruttando soprattutto il **cloud**
 - Un'evoluzione «light» delle architetture SOA (Service Oriented Application)
 - Le applicazioni che sono costituite da un certo numero di servizi indipendenti e distribuiti, ciascuno incentrato su un particolare aspetto, che comunicano tra loro per realizzare servizi più complessi.

Esempio MA



Software is changing, again

- Con AI e Coding Agents cambia il modo in cui:
 - produciamo codice
 - iteriamo sul software
 - distribuiamo attività tra umano e macchina
 - valutiamo la qualità
 - verifichiamo che il risultato sia corretto

Produrre software diventa più facile
Sapere se abbiamo prodotto il software giusto
diventa più difficile

L'AI riduce il costo della produzione di codice, ma aumenta l'importanza di intent, contesto, criteri di verifica e controllo del processo

Es Vibe Coding

Aggiungere a EasyLib la prenotazione di un libro non disponibile

- Prompt → Code → Correzione → Code → ...
 - `"Aggiungi la prenotazione dei libri"`
 - `"Gestisci due utenti sullo stesso libro"`
 - `"Aggiungi la cancellazione"`
 - `"Ora i test dei prestiti non passano: sistema"`
- Cosa succede?
 - le regole emergono durante l'implementazione;
 - parte del comportamento resta implicita nei prompt;
 - il contesto si disperde nella conversazione;
 - ogni correzione può introdurre regressioni;
 - non è chiaro quando la feature sia davvero "completa"
 - Il codice cresce. La specifica del comportamento resta implicita
- E se prima specificassimo cosa significa davvero "prenotare un libro"?

Rendere esplicito l'intento

Es: prenotare un libro non disponibile

Invece di partire dal codice, rendiamo esplicito il comportamento atteso:

- **Feature:** Book Reservation

- solo un utente autenticato può prenotare;
- il libro deve essere non disponibile;
- un utente non può avere due prenotazioni attive sullo stesso libro;
- le prenotazioni seguono un ordine FIFO;
- l'utente può cancellare la propria prenotazione;
- quando una copia torna disponibile, viene assegnata alla prima prenotazione valida.

- **Acceptance Criteria**

Given un libro senza copie disponibili

When un utente autenticato lo prenota

Then viene creata una prenotazione e l'utente entra nella coda.

**Prima di chiedere all'agente “come implementarlo”,
rendiamo esplicito “cosa deve fare”**

Vibe Coding: dove perdiamo il controllo?

Idea → Prompt → Code → Correzione → Code → ...

- **Intent implicito**
Le regole restano distribuite nei prompt.
- **Contesto fragile**
Decisioni e assunzioni possono andare perse.
- **Criteri di completezza poco chiari**
Quando possiamo dire che la feature è davvero finita?
- **Regressioni difficili da controllare**
Ogni correzione può rompere altro
- **Scarsa traceability**
È difficile collegare requisito → decisione → codice → test.

Il problema non è che l'AI scrive codice. Il problema è che l'intento e i criteri di verifica restano troppo impliciti.

- Come manteniamo la velocità del coding agent senza perdere il controllo ingegneristico?

Spec-Driven Development: trend emergente

Non esiste ancora un unico standard SDD, ma diversi strumenti e framework stanno convergendo su un pattern simile: **rendere esplicito l'intento prima di delegare l'implementazione all'AI**

Esempio	Idea / workflow
GitHub Spec Kit	Spec → Plan → Tasks → Implement → Converge
Kiro	Requirements → Design → Tasks → Implementation
Tessl	Requirements → Specs → Approval → Implement → Review → Verify
Thoughtworks Tech Radar	SDD descritto come approccio emergente nei workflow di AI-assisted coding

Pattern commune: Intent → Specification → Planning → Decomposition → Implementation → Verification

Meno prompt ad hoc, più artefatti espliciti e verificabili.

Spec-Driven Development (SDD)

Def: Approccio allo sviluppo in cui specification esplicite, persistenti e verificabili guidano pianificazione, implementazione e verifica

Intent → Spec → Plan → Tasks → Impl → Verification

- **Specification:** rende esplicito cosa deve fare il sistema
- **Plan & Tasks:** traducono la specification in lavoro tecnico eseguibile
- **Verification:** controlla che l'implementazione sia conforme alla specification

La specification non è solo documentazione: è un artefatto operativo del processo di sviluppo

Requirements ≠ Specification

Requirements Engineering WHAT & WHY

- obiettivi e bisogni
- RF / RNF
- User Story
- Use Case
- Acceptance Criteria
- mockup / User Flow
- vincoli
- linguaggio orientato agli stakeholder

Output: una descrizione del sistema principalmente human-oriented

AI-Ready Specification PRECISE WHAT

Trasformare i requisiti in una descrizione sufficientemente precisa da guidare implementazione e verifica

- feature ben delimitate
- comportamento atteso
- business rules
- condizioni e vincoli
- scenari principali ed eccezioni
- acceptance criteria verificabili
- ambiguità e GAP espliciti
- riferimenti ai requisiti originali

Output: una specification implementation-ready, senza decidere ancora necessariamente come implementarla

Dai Requirements alla Specification

Requirements → Feature Discovery + Clarification → Specification

- La specification non sostituisce i requirements: li raffina mantenendone la traceability

Requirement

- **RF12** — L'utente può prenotare un libro non disponibile

Specification

- Chi può prenotare?
- Quando è consentita la prenotazione?
- Quante prenotazioni può avere lo stesso utente?
- In quale ordine vengono servite?
- Cosa succede quando una copia viene restituita?
- Come si cancella una prenotazione?
- Quali errori devono essere gestiti?

RF12 esprime l'intento. La specification elimina le ambiguità necessarie per implementarlo e verificarlo.

Non ancora Technical Design

- La specification definisce principalmente cosa deve succedere. Database, API, classi, componenti e algoritmi appartengono al Technical Plan
- **WHAT → HOW → WORK**
 - **Specification**
WHAT — cosa deve fare il Sistema
 - **Technical Plan**
HOW — come lo realizziamo nel Sistema
 - **Tasks**
WORK — quali attività concrete devono essere eseguite

Feature

- Una feature è una **capacità coerente e riconoscibile** del sistema che fornisce un risultato significativo a un utente o stakeholder e che può essere specificata, implementata e verificata come unità
 - **Coerente**: Raggruppa comportamenti che appartengono allo stesso obiettivo
 - **Significativa**: Produce un risultato osservabile o un valore per un attore/stakeholder
 - **Delimitabile**: Possiamo stabilire cosa appartiene alla feature e cosa no
 - **Verificabile**: possiamo definire criteri per stabilire se è stata realizzata correttamente

Requirement ≠ Feature

Dai Requirements alle Features

Feature Discovery: individuare le unità significative di comportamento che dovremo poi specificare

- definizione operativa/didattica che adotteremo nel corso, perché “feature” non ha una granularità universalmente standardizzata

Feature Discovery

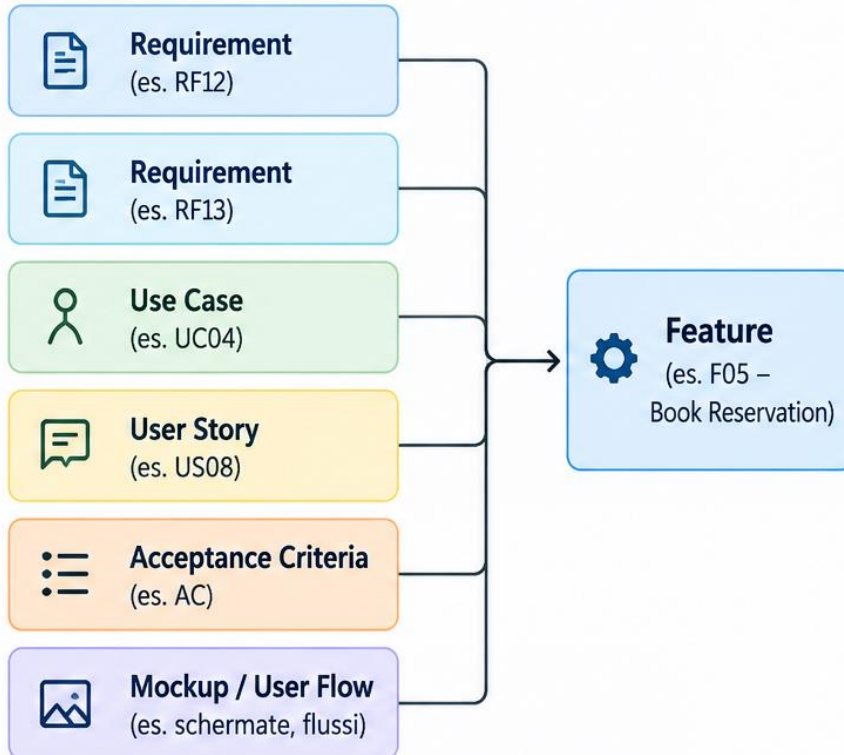
- analizza congiuntamente RF/RNF, Use Case, User Story, Acceptance Criteria, mockup, User Flow;
- individua feature, sovrapposizioni e duplicati;
- non assume che 1 requisito = 1 feature;
- collega ogni feature ai requisiti di origine

Clarification

- identifica ambiguità;
- rende esplicite assunzioni nascoste;
- evidenzia casi mancanti;
- segnala i GAP invece di inventare dettagli.

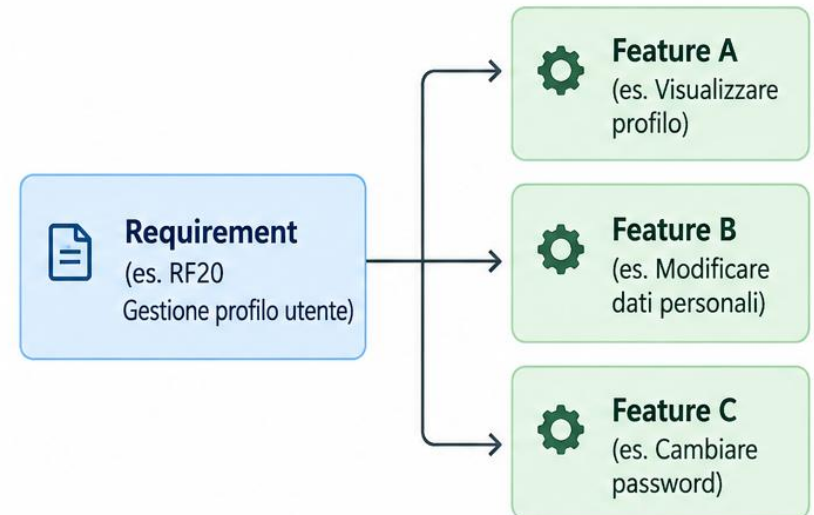
Molti artefatti del D1 → Una Feature

Una feature emerge dall'analisi congiunta di diversi artefatti.



Un Requirement → Più Feature

Un singolo requisito può contribuire a più feature.



Non assumiamo: 1 Requirement = 1 Feature

Le feature devono essere individuate analizzando congiuntamente tutti gli artefatti rilevanti del D1.

Feature Specification

Una scheda standard, semplice e focalizzata sul comportamento della feature.

D2
AI-Ready
Specification

Esempio: F05 – Book Reservation (EasyLib)

	1. Feature ID e nome	F05 – Book Reservation Gestione delle prenotazioni di libri non disponibili.
	2. Riferimenti D1	Source: RF12, RF13, UC04, US08, AC12.1–AC12.3
	3. Scopo	Consentire a un utente autenticato di prenotare un libro quando nessuna copia è disponibile.
	4. Comportamento atteso	Il sistema crea una prenotazione e la inserisce nella coda delle prenotazioni.
	5. Business Rules	• Un utente non può avere più di una prenotazione per lo stesso libro. • Le prenotazioni sono servite in ordine FIFO. • Solo l'utente che ha effettuato la prenotazione può cancellarla.
	6. Scenari principali	• Prenotazione corretta • Utente già prenotato • Libro nuovamente disponibile • Cancellazione della prenotazione
	7. Acceptance Criteria	AC-F05-01 Given un utente autenticato e nessuna copia disponibile When l'utente richiede la prenotazione Then viene creata una prenotazione e l'utente è inserito nella coda.
	8. GAP / Open Questions	GAP-F05-01: Il D1 non specifica per quanto tempo una copia assegnata rimane riservata.

Cosa include

- ✓ Comportamento osservabile
- ✓ Business rules
- ✓ Scenari principali (anche eccezioni)
- ✓ Acceptance criteria verificabili
- ✓ Riferimenti ai requisiti del D1
- ✓ GAP e domande aperte

Cosa non include

- ✗ Soluzioni tecniche specifiche
- ✗ API, database, classi
- ✗ Decisioni architetturali
- ✗ Dettagli implementativi (questo viene definito nel Technical Plan)

La Feature Specification descrive cosa deve fare il sistema, non come deve essere implementato.

Specification Quality Gate

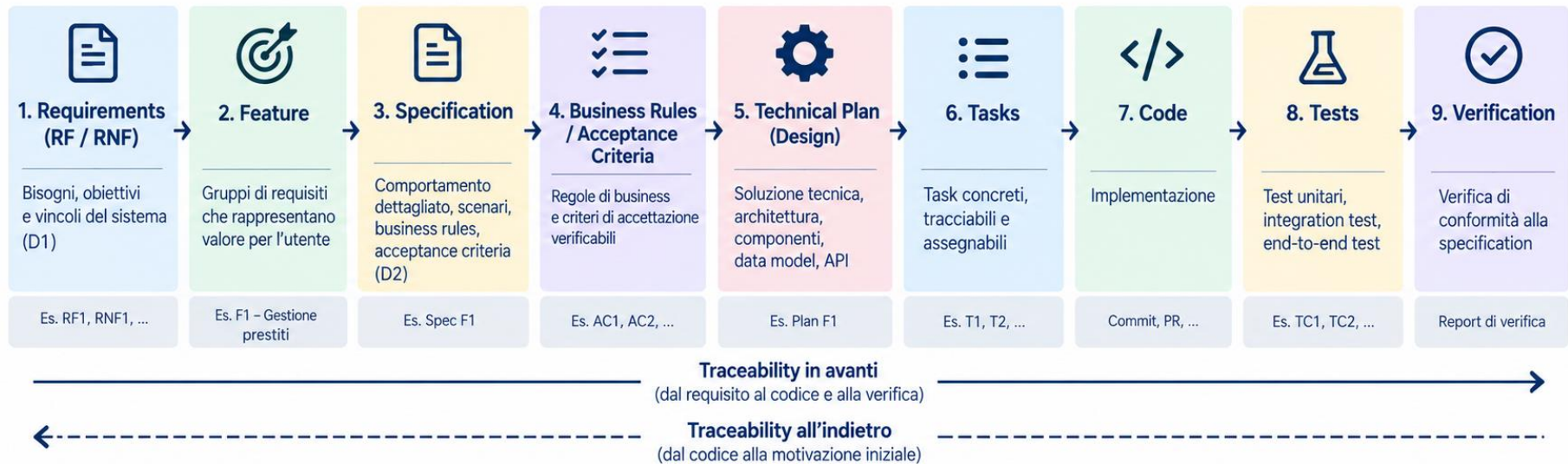
- Questa feature è abbastanza chiara da poter passare al Technical Plan?
 - **Completezza:** sono coperti comportamento principale, alternative ed errori rilevanti?
 - **Chiarezza:** ci sono termini ambigui o assunzioni implicite?
 - **Coerenza:** business rules, scenari e acceptance criteria sono compatibili tra loro?
 - **Verificabilità:** possiamo stabilire con test/evidenze se la feature è implementata correttamente?
 - **Traceability:** sappiamo da quali RF/RNF, Use Case, User Story o altri artefatti deriva?
- **PASS:** la feature può passare al **Technical Plan**
- **GAP / CLARIFY:** la feature torna a Clarification
- **Se una decisione necessaria manca, non la inventiamo: la segnaliamo come GAP**

Traceability end-to-end

Dal bisogno iniziale al codice, fino alla verifica.

Ogni artefatto deve poter essere ricondotto a ciò che lo ha motivato.

IS 2026
Spec-Driven
Development



Ogni artefatto deve poter essere ricondotto a ciò che lo ha motivato.

La traceability garantisce coerenza, facilita la manutenzione e supporta la verifica.

Human Gates: dove il team mantiene il controllo

L'AI può aiutare a produrre artefatti e codice, ma alcuni passaggi del processo richiedono una decisione esplicita del team.

IS 2026
Spec-Driven
Development



Domande chiave per ogni gate

- 1 La specification è sufficientemente chiara?**
 - comportamento principale, alternative, errori
 - business rules definite
 - nessuna ambiguità rilevante (o GAP espliciti)
- 2 Il piano tecnico è coerente?**
 - allineato alla specification
 - compatibile con l'architettura e i vincoli
 - soluzioni tecniche ragionevoli
- 3 I task sono adeguati?**
 - sufficientemente piccoli e chiari
 - tracciabili a feature e componenti
 - assegnabili (a persone o AI agents)
- 4 Il risultato è conforme?**
 - implementazione conforme alla specification
 - test superati
 - acceptance criteria soddisfatti



Delegare attività ≠ delegare responsabilità.

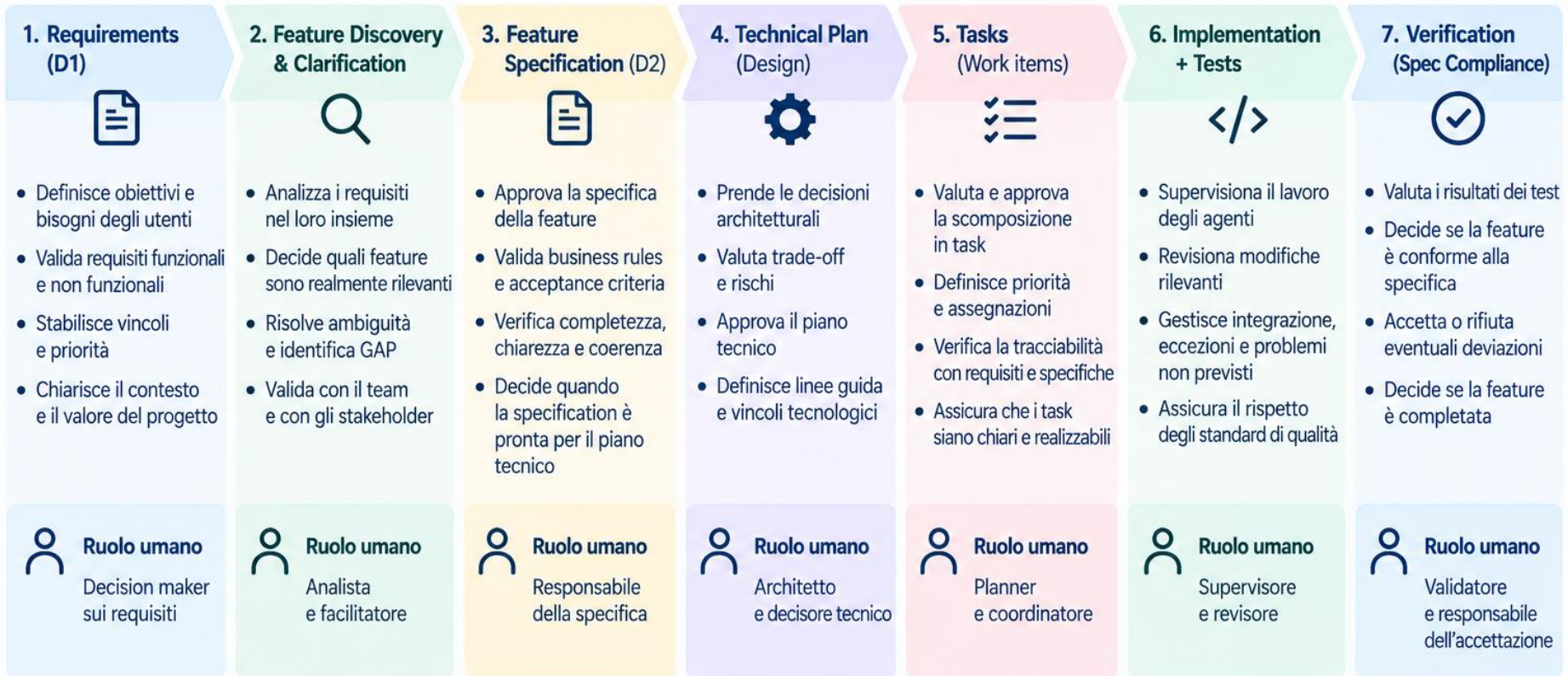
Il team decide quando un artefatto è abbastanza maturo da passare alla fase successiva.

Cosa fanno gli umani?

Gli agenti AI assistono. Il team governa il processo.

IS 2026

Spec-Driven
Development



L'umano non deve necessariamente scrivere tutto il codice.

Deve mantenere il controllo sulle decisioni, sui criteri di qualità e sull'accettazione del risultato.

AI executes and proposes.

Humans decide and are accountable.

AI Coding Agents nel processo IS 2026

L'AI può supportare il team in tutte le fasi, con ruoli diversi.
Il team mantiene il controllo sulle decisioni chiave.

IS 2026
Spec-Driven
Development



L'AI può contribuire in quasi tutte le fasi.

Il livello di delega cambia, la responsabilità no.



AI come amplificatore delle capacità del team, non come sostituto delle decisioni.

AI Agents: cosa dare in input e cosa producono in output

Per ogni fase del processo, fornisci all'agente gli artefatti, il contesto e le istruzioni appropriate.
L'agente produce un output che il team revisiona e approva.

IS 2026

Spec-Driven
Development



Input chiari → Output utili

Più l'input è preciso e contestualizzato, più l'agente produce risultati di qualità.



Buone pratiche

- Fornisci sempre il contesto rilevante
- Usa template e linee guida
- Specifica il formato desiderato dell'output
- Revisiona sempre l'output prima di procedere

SSD Takeaways

- AI Coding Agents accelerano la produzione di software
- La velocità aumenta il bisogno di specifiche, verificabilità e controllo
- SDD rende specification, plan, tasks e verification artefatti espliciti
- L'AI può contribuire in molte fasi, ma il team resta responsabile delle decisioni e dell'accettazione finale